

Guía Completa: Deploy de SmartSolutions VE en DigitalOcean

Tabla de Contenidos

1. Requisitos Previos
 2. Fase 1: Crear y Configurar el Droplet
 3. Fase 2: Configuración Inicial del Servidor
 4. Fase 3: Instalar Docker y Docker Compose
 5. Fase 4: Configurar PostgreSQL
 6. Fase 5: Desplegar la Aplicación Django
 7. Fase 6: Configurar Nginx como Reverse Proxy
 8. Fase 7: SSL/HTTPS con Let's Encrypt
 9. Fase 8: Configuración de Dominio
 10. Fase 9: Monitoreo y Mantenimiento
 11. Comandos de Referencia Rápida
 12. Troubleshooting
-

Requisitos Previos

Lo que necesitas tener listo:

-  Cuenta en DigitalOcean (usa [este link con \\$200 crédito](#))
-  Tu dominio registrado (ej: smartsolutions.com.ve)
-  Proyecto Django de SmartSolutions VE (el ZIP que descargaste)
-  Cuenta en Resend.com para emails (plan gratuito)
-  Terminal SSH en tu computadora (Linux/Mac nativo, Windows usa WSL2 o PuTTY)

Costo mensual estimado:

- Droplet básico: **\$6/mes** (1GB RAM, 1 CPU, 25GB SSD)
 - Dominio: Ya lo tienes
 - SSL: Gratis (Let's Encrypt)
 - **Total: ~\$6/mes ✨**
-

Fase 1: Crear y Configurar el Droplet

1.1 Crear el Droplet

1. **Inicia sesión en DigitalOcean** → <https://cloud.digitalocean.com>
2. **Create** → **Droplets**
3. **Configuración del Droplet:**

Choose Region:

- Elige el más cercano a Venezuela:
 - New York (mejor latencia para VE)
 - Miami (alternativa)

Choose an image:

- Ubuntu 24.04 LTS x64

Choose Size:

- Basic Plan
- Regular CPU
- \$6/mo - 1GB RAM / 1 CPU / 25GB SSD / 1TB transfer

Choose Authentication Method:

- SSH Keys (RECOMENDADO - más seguro)
- O
- Password (más fácil para empezar)

Hostname:

- smartsolutions-prod

4. Si elegiste SSH Keys:

bash

En tu computadora local, genera una llave SSH si no tienes:

```
ssh-keygen -t ed25519 -C "tu@email.com"
```

Copia el contenido de la llave pública:

```
cat ~/.ssh/id_ed25519.pub
```

Pega el contenido en DigitalOcean → Add SSH Key

5. **Create Droplet** → Espera 1-2 minutos

6. **Copia la IP pública** del droplet (algo como `164.92.XXX.XXX`)

Fase 2: Configuración Inicial del Servidor

2.1 Conectarte por SSH

```
bash

# Con contraseña:
ssh root@TU_IP_PUBLICA

# Con SSH key:
ssh -i ~/.ssh/id_ed25519 root@TU_IP_PUBLICA

# Primera vez te preguntará si confías en el servidor, escribe: yes
```

2.2 Actualizar el sistema

```
bash

apt update && apt upgrade -y
```

2.3 Crear un usuario no-root (seguridad)

```
bash

# Crear usuario
adduser deploy
# Te pedirá contraseña - usa una segura y guárdala

# Darle permisos de sudo
usermod -aG sudo deploy

# Cambiar a ese usuario
su - deploy
```

2.4 Configurar firewall básico

```
bash
```

```
# Permitir SSH
sudo ufw allow OpenSSH
```

```
# Permitir HTTP y HTTPS (para el sitio web)
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

```
# Activar firewall
sudo ufw enable
```

```
# Verificar estado
sudo ufw status
```

Fase 3: Instalar Docker y Docker Compose

3.1 Instalar Docker

```
bash

# Instalar dependencias
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common

# Agregar GPG key de Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Agregar repositorio de Docker
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalar Docker
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io

# Agregar tu usuario al grupo docker (para no usar sudo)
sudo usermod -aG docker $USER

# Aplicar cambios (o cierra sesión y vuelve a entrar)
newgrp docker

# Verificar instalación
docker --version
```

3.2 Instalar Docker Compose

```
bash

# Descargar Docker Compose
sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose

# Dar permisos de ejecución
sudo chmod +x /usr/local/bin/docker-compose

# Verificar instalación
docker-compose --version
```

Fase 4: Configurar PostgreSQL

4.1 Crear estructura de directorios

```
bash

# Crear directorio para el proyecto
mkdir -p ~/smartsolutions
cd ~/smartsolutions

# Crear directorio para datos de PostgreSQL
mkdir -p ~/smartsolutions/postgres-data
```

4.2 Crear archivo Docker Compose inicial

```
bash

nano docker-compose.yml
```

Pega este contenido:

```
yml
```

```
version: '3.8'

services:
  db:
    image: postgres:16-alpine
    container_name: smartsolutions_db
    restart: always
    environment:
      POSTGRES_DB: smartsolutions
      POSTGRES_USER: smartsolutions
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - ./postgres-data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    networks:
      - smartsolutions_network

networks:
  smartsolutions_network:
    driver: bridge
```

Guarda con **Ctrl + O**, Enter, **Ctrl + X**

4.3 Crear archivo .env con credenciales

```
bash

nano .env
```

Pega este contenido (cambia los valores):

```
bash
```

```
# Django
SECRET_KEY=genera-una-clave-secreta-aqui-de-50-caracteres-random
DEBUG=False
ALLOWED_HOSTS=tu-dominio.com,www.tu-dominio.com,TU_IP_PUBLICA

# Base de datos
DB_NAME=smartsolutions
DB_USER=smartsolutions
DB_PASSWORD=cambia-esto-por-una-contraseña-segura
DB_HOST=db
DB_PORT=5432

# Email (Resend)
RESEND_API_KEY=re_tu_api_key_de_resend
DEFAULT_FROM_EMAIL=noreply@tu-dominio.com
CONTACT_EMAIL=tu@email.com

# Sitio
SITE_URL=https://tu-dominio.com
```

IMPORTANTE: Genera una `SECRET_KEY` segura:

```
bash

python3 -c 'from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())'
```

Guarda con `Ctrl + O`, Enter, `Ctrl + X`

4.4 Levantar PostgreSQL

```
bash

docker-compose up -d db

# Verificar que está corriendo
docker-compose ps
```

Fase 5: Desplegar la Aplicación Django

5.1 Subir el proyecto al servidor

Desde **tu computadora local**:

```
bash
```

```
# Comprimir el proyecto (asegúrate de estar en la carpeta correcta)
```

```
cd /ruta/donde/descomprimiste/smartsolutions
```

```
zip -r smartsolutions.zip . -x "*.pyc" -x "__pycache__/*" -x "venv/*" -x ".git/*"
```

```
# Subir al servidor vía SCP
```

```
scp smartsolutions.zip deploy@TU_IP_PUBLICA:~/smartsolutions/
```

```
# Conectarte de nuevo al servidor
```

```
ssh deploy@TU_IP_PUBLICA
```

En el **servidor**:

```
bash
```

```
cd ~/smartsolutions
```

```
# Descomprimir
```

```
unzip smartsolutions.zip -d app/
```

```
# Verificar
```

```
ls app/
```

```
# Deberías ver: apps/, config/, manage.py, requirements.txt, etc.
```

5.2 Crear Dockerfile para Django

```
bash
```

```
nano app/Dockerfile
```

Pega este contenido:

```
dockerfile
```



```
FROM python:3.11-slim

# Variables de entorno
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

# Directorio de trabajo
WORKDIR /app

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y \
    gcc \
    postgresql-client \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Copiar requirements e instalar
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar el proyecto
COPY . .

# Crear directorio para static files
RUN mkdir -p /app/staticfiles

# Recopilar archivos estáticos
RUN python manage.py collectstatic --noinput

# Exponer puerto
EXPOSE 8000

# Comando de inicio
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]
```

Guarda con **Ctrl + O**, Enter, **Ctrl + X**

5.3 Actualizar docker-compose.yml

```
bash
cd ~/smartsolutions
nano docker-compose.yml
```

Reemplaza TODO el contenido con:

yaml

version: '3.8'

services:

db:

image: postgres:16-alpine

container_name: smartsolutions_db

restart: always

environment:

POSTGRES_DB: \${DB_NAME}

POSTGRES_USER: \${DB_USER}

POSTGRES_PASSWORD: \${DB_PASSWORD}

volumes:

- ./postgres-data:/var/lib/postgresql/data

networks:

- smartsolutions_network

web:

build: ./app

container_name: smartsolutions_web

restart: always

env_file:

- .env

volumes:

- ./app:/app

- static_volume:/app/staticfiles

- media_volume:/app/media

depends_on:

- db

networks:

- smartsolutions_network

command: >

sh -c "python manage.py migrate &&

python manage.py collectstatic --noinput &&

gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3"

nginx:

image: nginx:alpine

container_name: smartsolutions_nginx

restart: always

ports:

- "80:80"

- "443:443"

volumes:

- ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
- ./nginx/conf.d:/etc/nginx/conf.d:ro
- static_volume:/app/staticfiles:ro
- media_volume:/app/media:ro
- ./certbot/conf:/etc/letsencrypt:ro
- ./certbot/www:/var/www/certbot:ro

depends_on:

- web

networks:

- smartsolutions_network

volumes:

static_volume:

media_volume:

networks:

smartsolutions_network:

driver: bridge

Guarda con **Ctrl + O**, Enter, **Ctrl + X**

5.4 Construir y levantar la aplicación

bash

Construir la imagen de Django

`docker-compose build web`

Levantar todos los servicios

`docker-compose up -d`

Ver logs para verificar que todo está OK

`docker-compose logs -f web`

Si ves errores, presiona Ctrl+C y revisa

5.5 Crear superusuario de Django

bash

```
docker-compose exec web python manage.py createsuperuser
```

```
# Te pedirá:
```

```
# - Username: admin
```

```
# - Email: tu@email.com
```

```
# - Password: (elige una segura)
```

Fase 6: Configurar Nginx como Reverse Proxy

6.1 Crear estructura de Nginx

```
bash
```

```
mkdir -p ~/smartsolutions/nginx/conf.d
```

6.2 Crear configuración de Nginx

```
bash
```

```
nano ~/smartsolutions/nginx/nginx.conf
```

Pega este contenido:

```
nginx
```

```

user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;
    client_max_body_size 20M;

    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_types text/plain text/css text/xml text/javascript
        application/json application/javascript application/xml+rss
        application/rss+xml font/truetype font/opentype
        application/vnd.ms-fontobject image/svg+xml;

    include /etc/nginx/conf.d/*.conf;
}

```

Guarda con **Ctrl + O**, Enter, **Ctrl + X**

6.3 Crear configuración del sitio (sin SSL por ahora)

```
bash
```

`nano ~/smartsolutions/nginx/conf.d/smartsolutions.conf`

Pega este contenido (cambia `tu-dominio.com`):

```
nginx

server {
    listen 80;
    server_name tu-dominio.com www.tu-dominio.com;

    # Para Let's Encrypt
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    # Redirigir todo a HTTPS (lo activaremos después de obtener SSL)
    # location / {
    #     return 301 https://$host$request_uri;
    # }

    # Por ahora, servir directamente (temporal)
    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        alias /app/staticfiles/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /media/ {
        alias /app/media/;
        expires 7d;
    }
}
```

Guarda con `Ctrl + O`, Enter, `Ctrl + X`

6.4 Reiniciar Nginx

```
bash

docker-compose restart nginx

# Verificar logs
docker-compose logs nginx
```

Fase 7: SSL/HTTPS con Let's Encrypt

7.1 Crear estructura para Certbot

```
bash

mkdir -p ~/smartsolutions/certbot/{conf,www}
```

7.2 Obtener certificado SSL

IMPORTANTE: Antes de ejecutar esto, asegúrate de que:

- Tu dominio apunta a la IP del servidor (ver Fase 8)
- Nginx está corriendo y respondiendo en el puerto 80

```
bash

# Obtener certificado SSL (cambia tu-dominio.com y tu@email.com)
docker run -it --rm \
  -v ~/smartsolutions/certbot/conf:/etc/letsencrypt \
  -v ~/smartsolutions/certbot/www:/var/www/certbot \
  certbot/certbot certonly \
  --webroot \
  --webroot-path=/var/www/certbot \
  --email tu@email.com \
  --agree-tos \
  --no-eff-email \
  -d tu-dominio.com \
  -d www.tu-dominio.com

# Si todo sale bien, verás: "Successfully received certificate"
```

7.3 Actualizar configuración de Nginx con HTTPS


```
bash
```

```
nano ~/smartsolutions/nginx/conf.d/smartsolutions.conf
```

Reemplaza TODO el contenido con:

```
nginx
```

```
# Redirigir HTTP a HTTPS
```

```
server {  
    listen 80;  
    server_name tu-dominio.com www.tu-dominio.com;  
  
    location /.well-known/acme-challenge/ {  
        root /var/www/certbot;  
    }  
  
    location / {  
        return 301 https://$host$request_uri;  
    }  
}
```

```
# Servidor HTTPS
```

```
server {  
    listen 443 ssl http2;  
    server_name tu-dominio.com www.tu-dominio.com;  
  
    # Certificados SSL  
    ssl_certificate /etc/letsencrypt/live/tu-dominio.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/tu-dominio.com/privkey.pem;
```

```
# Configuración SSL moderna
```

```
ssl_protocols TLSv1.2 TLSv1.3;  
ssl_ciphers HIGH:!aNULL:!MD5;  
ssl_prefer_server_ciphers on;  
ssl_session_cache shared:SSL:10m;  
ssl_session_timeout 10m;
```

```
# Headers de seguridad
```

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;  
add_header X-Frame-Options "SAMEORIGIN" always;  
add_header X-Content-Type-Options "nosniff" always;  
add_header X-XSS-Protection "1; mode=block" always;
```

```
# Django backend
```

```
location / {  
    proxy_pass http://web:8000;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;
```

```
# Timeouts
proxy_connect_timeout 60s;
proxy_send_timeout 60s;
proxy_read_timeout 60s;
}

# Static files
location /static/ {
    alias /app/staticfiles/;
    expires 30d;
    add_header Cache-Control "public, immutable";
    access_log off;
}

# Media files
location /media/ {
    alias /app/media/;
    expires 7d;
    add_header Cache-Control "public";
}

# Limits
client_max_body_size 20M;
}
```

Guarda y reinicia Nginx:

```
bash

docker-compose restart nginx

# Verificar que no hay errores
docker-compose logs nginx
```

7.4 Configurar renovación automática de SSL

```
bash

# Crear script de renovación
nano ~/smartsolutions/renew-ssl.sh
```

Pega este contenido:

```
bash
```

```
#!/bin/bash
```

```
cd /home/deploy/smartsolutions
```

```
docker run --rm \
-v $PWD/certbot/conf:/etc/letsencrypt \
-v $PWD/certbot/www:/var/www/certbot \
certbot/certbot renew
```

```
docker-compose restart nginx
```

Haz el script ejecutable y configura cron:

```
bash
```

```
chmod +x ~/smartsolutions/renew-ssl.sh
```

```
# Editar crontab
```

```
crontab -e
```

```
# Agrega esta línea al final (se ejecutará cada día a las 3am):
```

```
0 3 * * * /home/deploy/smartsolutions/renew-ssl.sh >> /home/deploy/smartsolutions/ssl-renew.log 2>&1
```

Fase 8: Configuración de Dominio

8.1 Configurar DNS en tu proveedor de dominio

Ve al panel de tu proveedor de dominio (GoDaddy, Namecheap, etc.) y crea estos registros DNS:

Tipo: A

Nombre: @

Valor: TU_IP_PUBLICA_DEL_DROPLET

TTL: 3600

Tipo: A

Nombre: www

Valor: TU_IP_PUBLICA_DEL_DROPLET

TTL: 3600

NOTA: Los cambios de DNS pueden tardar de 5 minutos a 48 horas en propagarse.

8.2 Verificar propagación

```
bash

# Desde tu computadora local:
nslookup tu-dominio.com
nslookup www.tu-dominio.com

# Deberían mostrar la IP de tu droplet
```

Fase 9: Monitoreo y Mantenimiento

9.1 Ver logs en tiempo real

```
bash

# Logs de todos los servicios
docker-compose logs -f

# Solo logs de Django
docker-compose logs -f web

# Solo logs de Nginx
docker-compose logs -f nginx

# Solo logs de PostgreSQL
docker-compose logs -f db
```

9.2 Comandos útiles de mantenimiento

```
bash
```

Reiniciar un servicio específico

`docker-compose restart web`

`docker-compose restart nginx`

Reiniciar todo

`docker-compose restart`

Ver estado de los servicios

`docker-compose ps`

Entrar a un contenedor

`docker-compose exec web bash`

`docker-compose exec db psql -U smartsolutions -d smartsolutions`

Ver uso de recursos

`docker stats`

Limpiar imágenes no usadas (libera espacio)

`docker system prune -a`

9.3 Backup de la base de datos

`bash`

Crear script de backup

`nano ~/backup-db.sh`

Pega este contenido:

`bash`

```
#!/bin/bash
```

```
BACKUP_DIR=/home/deploy/backups
```

```
mkdir -p $BACKUP_DIR
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```
BACKUP_FILE=$BACKUP_DIR/smartsolutions_$DATE.sql
```

```
docker-compose exec -T db pg_dump -U smartsolutions smartsolutions > $BACKUP_FILE
```

```
# Comprimir
```

```
gzip $BACKUP_FILE
```

```
# Mantener solo los últimos 7 backups
```

```
ls -t $BACKUP_DIR/*.sql.gz | tail -n +8 | xargs rm -f
```

```
echo "Backup creado: $BACKUP_FILE.gz"
```

Haz el script ejecutable y configura cron:

```
bash
```

```
chmod +x ~/backup-db.sh
```

```
# Agregar a crontab (backup diario a las 2am)
```

```
crontab -e
```

```
# Agregar:
```

```
0 2 * * * /home/deploy/backup-db.sh >> /home/deploy/backup.log 2>&1
```

9.4 Restaurar un backup

```
bash
```

```
# Descomprimir el backup
```

```
gunzip ~/backups/smartsolutions_FECHA.sql.gz
```

```
# Restaurar
```

```
docker-compose exec -T db psql -U smartsolutions smartsolutions < ~/backups/smartsolutions_FECHA.sql
```

Comandos de Referencia Rápida

Acceso SSH

```
bash  
ssh deploy@TU_IP
```

Docker Compose

```
bash  
  
cd ~/smartsolutions  
  
# Levantar servicios  
docker-compose up -d  
  
# Detener servicios  
docker-compose down  
  
# Reconstruir después de cambios en código  
docker-compose build web  
docker-compose up -d --force-recreate web  
  
# Ver logs  
docker-compose logs -f web
```

Django

```
bash  
  
# Ejecutar comandos de Django  
docker-compose exec web python manage.py makemigrations  
docker-compose exec web python manage.py migrate  
docker-compose exec web python manage.py createsuperuser  
docker-compose exec web python manage.py collectstatic --noinput
```

Base de datos

```
bash
```



```
# Conectar a PostgreSQL
```

```
docker-compose exec db psql -U smartsolutions -d smartsolutions
```

```
# Backup
```

```
docker-compose exec -T db pg_dump -U smartsolutions smartsolutions > backup.sql
```

Troubleshooting

Problema: "Connection refused" al visitar el sitio

Diagnóstico:

```
bash
```

```
# Verificar que los contenedores están corriendo
```

```
docker-compose ps
```

```
# Ver logs de Nginx
```

```
docker-compose logs nginx
```

```
# Ver logs de Django
```

```
docker-compose logs web
```

Solución:

- Si el contenedor web no está corriendo: `docker-compose up -d web`
- Si hay errores en logs: corregir y `docker-compose restart`

Problema: Error 502 Bad Gateway

Causa: Nginx no puede conectar con Django

Solución:

```
bash
```

```
# Verificar que Django está corriendo
docker-compose exec web curl http://localhost:8000
```

```
# Si no responde, revisar logs
docker-compose logs web
```

```
# Reiniciar Django
docker-compose restart web
```

Problema: Archivos estáticos no se muestran (CSS/JS)

Solución:

```
bash

# Recopilar archivos estáticos
docker-compose exec web python manage.py collectstatic --noinput

# Reiniciar Nginx
docker-compose restart nginx

# Verificar permisos
docker-compose exec web ls -la /app/staticfiles/
```

Problema: No puedo conectarme por SSH

Solución:

```
bash

# Verificar que el firewall permite SSH
sudo ufw status

# Si no permite, agregar:
sudo ufw allow 22/tcp
sudo ufw reload

# Verificar que el servicio SSH está corriendo
sudo systemctl status sshd
```

Problema: Certificado SSL no se renueva automáticamente

Diagnóstico:

```
bash

# Ver logs de renovación
cat ~/smartsolutions/ssl-renew.log

# Probar renovación manual
~/smartsolutions/renew-ssl.sh
```

Solución:

- Verificar que el cron está configurado: `crontab -l`
 - Verificar que el script tiene permisos: `ls -la ~/smartsolutions/renew-ssl.sh`
-

Problema: Espacio en disco lleno

Diagnóstico:

```
bash

# Ver uso de disco
df -h

# Ver tamaño de imágenes Docker
docker system df
```

Solución:

```
bash

# Limpiar contenedores e imágenes no usadas
docker system prune -a

# Limpiar logs viejos
sudo journalctl --vacuum-time=7d

# Limpiar backups viejos
rm ~/backups/*_old.sql.gz
```



Tu sitio debería estar funcionando en:

- **HTTP:** <http://tu-dominio.com> (redirige a HTTPS)
- **HTTPS:** <https://tu-dominio.com> ✓
- **Admin:** <https://tu-dominio.com/admin>

Checklist final:

- ☐ El sitio carga correctamente en HTTPS
 - ☐ Los archivos estáticos (CSS/JS) se ven bien
 - ☐ Puedes acceder al admin Django
 - ☐ El formulario de contacto funciona y envía emails
 - ☐ El certificado SSL está activo (candado verde en el navegador)
 - ☐ Los backups automáticos están configurados
-

Próximos pasos:

1. **Configurar Google Analytics** en `templates/base.html`
 2. **Agregar contenido** desde el admin (Servicios, Testimonios, Casos)
 3. **Configurar CI/CD** con GitHub Actions (automatizar deploys)
 4. **Monitoreo:** Instalar Uptime Kuma o similar
 5. **Performance:** Activar Redis para caché (opcional)
-

¿Tienes problemas? Revisa la sección de Troubleshooting o busca los logs con `docker-compose logs -f`